Lab: Neural Network

65090502417 ปิยพัทธ์ จันทร์ทวี

2025-10-31

```
library(ISLR2)
```

```
## Warning: package 'ISLR2' was built under R version 4.4.3
```

```
library(torch)
```

```
## Warning: package 'torch' was built under R version 4.4.3
```

```
library(luz) # high-level interface for torch
library(torchvision) # for datasets and image transformation
```

```
## Warning: package 'torchvision' was built under R version 4.4.3
```

```
library(torchdatasets) # for datasets we are going to use
```

```
## Warning: package 'torchdatasets' was built under R version 4.4.3
```

```
library(zeallot)
```

```
## Warning: package 'zeallot' was built under R version 4.4.3
```

```
Gitters <- na.omit(Hitters)
n <- nrow(Gitters)
set.seed(13)
ntest <- trunc(n / 3)
testid <- sample(1:n, ntest)

x <- scale(model.matrix(Salary ~ . - 1, data = Gitters))
y <- Gitters$Salary
```

```
torch_manual_seed(13)
###

###
modnn <- nn_module(
  initialize = function(input_size) {
    self$hidden <- nn_linear(input_size, 50)
    self$activation <- nn_relu()
    self$dropout <- nn_dropout(0.4)
    self$output <- nn_linear(50, 1)
  },
  forward = function(x) {
    x %>%
      self$hidden() %>%
      self$activation() %>%
      self$dropout() %>%
      self$output()
  }
)
###

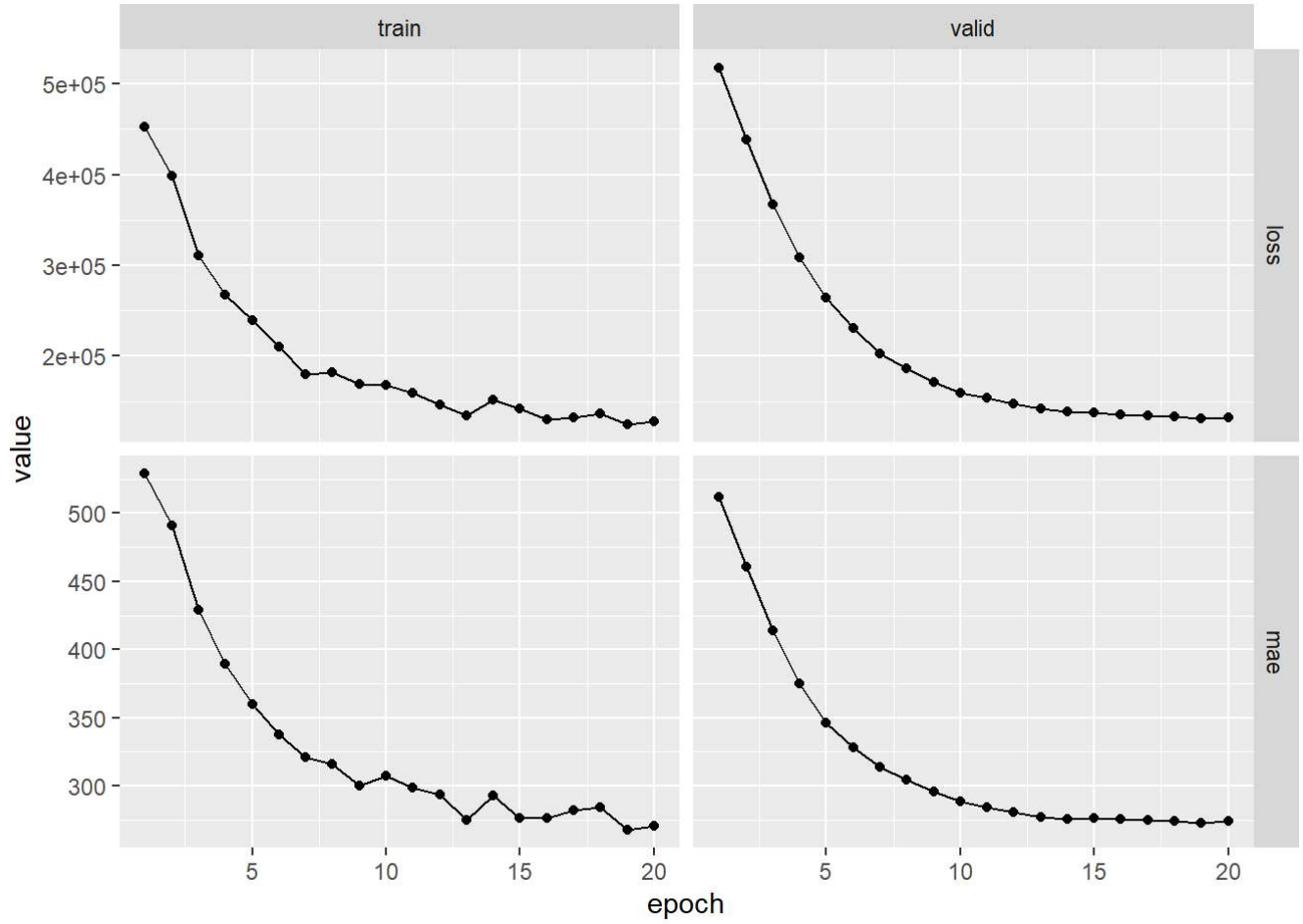
###
x <- scale(model.matrix(Salary ~ . - 1, data = Gitters))
###


###
x <- model.matrix(Salary ~ . - 1, data = Gitters) %>% scale()
###

###
modnn <- modnn %>%
  setup(
    loss = nn_mse_loss(),
    optimizer = optim_rmsprop,
    metrics = list(luz_metric_mae())
  ) %>%
  set_hparams(input_size = ncol(x))
###

###
fitted <- modnn %>%
  fit(
    data = list(x[-testid, ], matrix(y[-testid], ncol = 1)),
    valid_data = list(x[testid, ], matrix(y[testid], ncol = 1)),
    epochs = 20
  )
###

###
plot(fitted)
```




```
###
###
npred <- predict(fitted, x[testid, ])
mean(abs(y[testid] - as.numeric(npred)))
```

```
## [1] 274.0331
```

เพิ่ม Layer เป็น 3 Layers

```
#Lab add more layer
torch_manual_seed(13)

modnn_2 <- nn_module(
  initialize = function(input_size) {
    self$hidden_1 <- nn_linear(input_size, 50)
    self$activation_1 <- nn_relu()
    self$dropout_1 <- nn_dropout(0.4)

    self$hidden_2 <- nn_linear(50, 25)
    self$activation_2 <- nn_relu()
    self$dropout_2 <- nn_dropout(0.5)

    self$hidden_3 <- nn_linear(25, 5)
    self$activation_3 <- nn_relu()
    self$dropout_3 <- nn_dropout(0.2)


    self$output <- nn_linear(5, 1)
  },
  forward = function(x) {
    x %>%
      self$hidden_1() %>%
      self$activation_1() %>%
      self$dropout_1() %>%

      self$hidden_2() %>%
      self$activation_2() %>%
      self$dropout_2() %>%

      self$hidden_3() %>%
      self$activation_3() %>%
      self$dropout_3() %>%

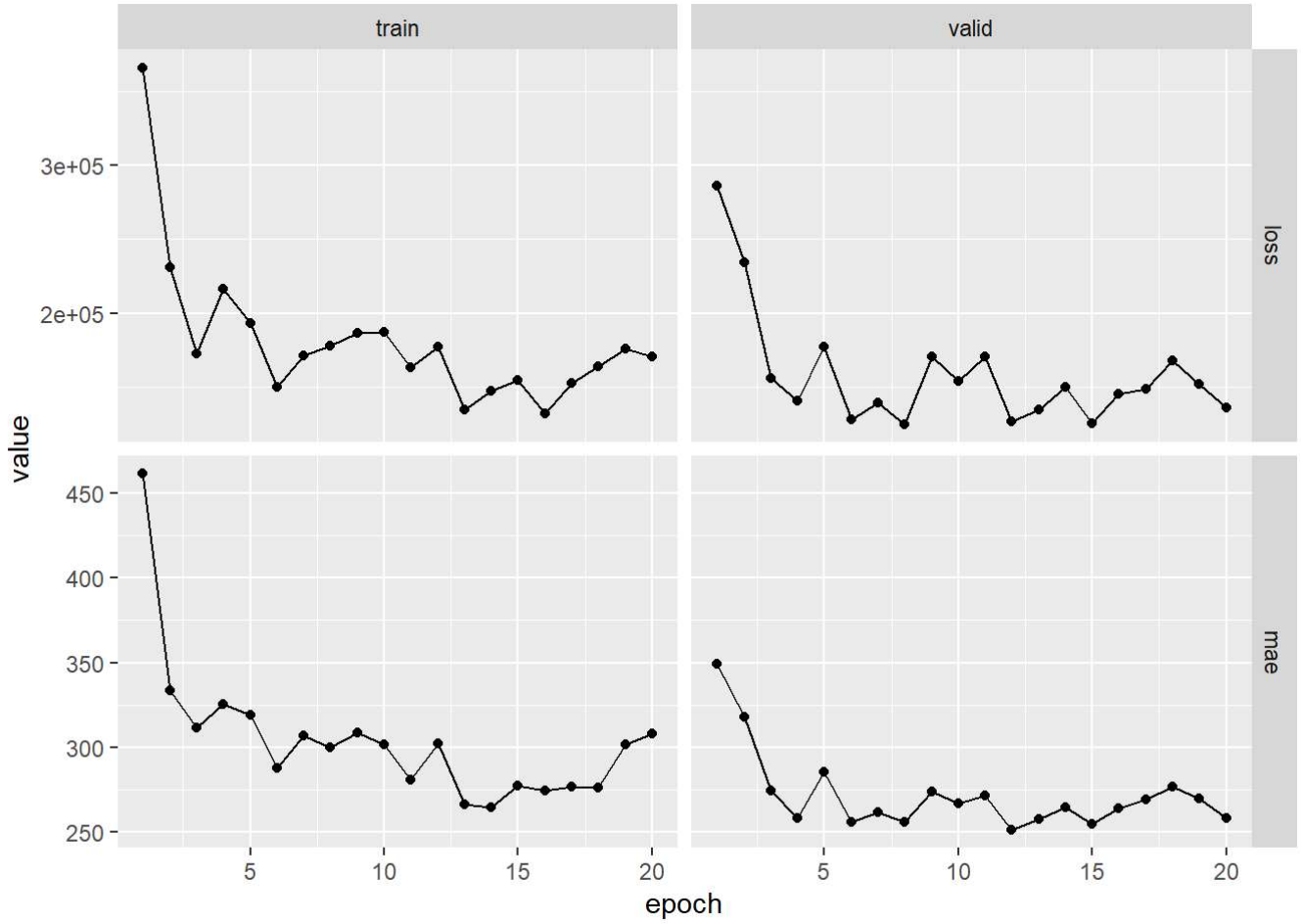
      self$output()
  }
)

modnn_2 <- modnn_2%>%
  setup(
    loss = nn_mse_loss(),
    optimizer = optim_rmsprop,
    metrics = list(luz_metric_mae())
  ) %>%
  set_hparams(input_size = ncol(x))
###

###
fitted_2 <- modnn_2 %>%
  fit(
    data = list(x[-testid, ], matrix(y[-testid], ncol = 1)),
    valid_data = list(x[testid, ], matrix(y[testid], ncol = 1)),
    epochs = 20
```

```
  )
###

###
plot(fitted_2)
```




```
###


###
npred_2 <- predict(fitted_2, x[testid, ])
mean(abs(y[testid] - as.numeric(npred_2)))
```

```
## [1] 257.8428
```

####################

Compare 1 Layer (50 -> 1, drop = 0.4) VS 3 Layers (50 -> 25 -> 5 -> 1, drop = 0.4 -> 0.5 -> 0.2)

```
mean(abs(y[testid] - as.numeric(npred)))
```

```
## [1] 274.0331
```

```
mean(abs(y[testid] - as.numeric(npred_2)))
```

```
## [1] 257.8428
```

สรุป: ได้ว่า การ เพิ่ม Layer สามารถเพิ่มประสิทธิภาพในการทำนายได้มากยิ่งขึ้น ด้วยผล MAE ที่ ลดลง